

When Does the Record of a Computation Matter?

Witness structure across substrates: proofs, edits, and circuits

Sean Mahdavian

Research Report (working draft)

Abstract

Every real computation makes two things: a result, and a record of how it got there. I call that record the witness. This paper asks a simple question: when is it worth keeping the witness? I tested three systems. First, code edits. I trained a small language model on 10,925 edits from the Mathlib library. Training with the edit itself (the diff) worked no better than training with just the before and after files. The diff is easy to compute from the two files, so writing it out adds nothing (Part I). Second, quantum circuits. I ran four pairs of circuits on IBM's `ibm_fez` chip. In each pair, a standard circuit and a ZX-optimized version compute the same function, and a machine proof of equality checks this before they run. The optimized circuits scored up to +0.067 higher in exact-answer probability. But the noise model predicted the wrong ranking, and careful controls show the gain comes from shorter run time, not fewer gates (Part II). Third, proofs. When the task is to guess a Lean proof step from the states before and after it, recovering the step is search, not reading. Here, training with the proof steps beat training with endpoints alone, by +0.024 to +0.050 across three seeds. Most of that gain, 68 to 76%, comes from the content of the steps, not their format (Part III). I then tested the natural guess about what drives this: the number of witnesses per endpoint pair, a quantity I defined formally in Lean. Exact computation kills the guess. The achievable gains collapse at both ends of the scale, so no simple law of this kind can exist for the family I tested (Part IV). Last, I asked whether more endpoint data could buy the same advantage. Across a $32\times$ range of budgets, the gap does not close. Extrapolation puts the zero point near eight million examples (Part V). I pre-registered all twelve protocols with timestamped commits and recorded every verdict before looking at the data; a public script, `verify.py`, recomputes every table. The conclusion, sized carefully: witness supervision transfers something that endpoint data cannot buy at any budget tested, and the naive way to predict when this happens is provably wrong.

1 Introduction

Every real computation makes two things: a result, and a record of how it got there. I call that record the *witness*. A proof has a witness: the steps. A code edit has one: the diff. A quantum computation has one: the circuit. Most machine learning pipelines keep only the results and throw the witness away. The silent bet is that anything the witness could teach can be recomputed from the results later. When is that bet safe? This paper measures the answer in three places: code edits, quantum circuits, and formal proofs.

The key question is how hard the witness is to recover from the endpoints. Sometimes recovery is easy. Given two versions of a file, a computer can work out the diff in polynomial time. Sometimes it is hard. Given the state of a Lean proof before and after one step, finding the step means search, while checking it is fast. If the endpoints determine the witness, then training on the witness should add nothing to training on the endpoints. If they do not, the witness carries information the endpoints lack, and training on it should help. Testing the “if” is the point of Parts I and III: Part I tests a domain where recovery is provably cheap, and Part III tests one where recovery is search. One disclaimer belongs here: this find-versus-check gap has the same shape as the definition of NP. This paper says nothing about P versus NP. It only sorts domains by how expensive recovery is, and nothing later should be read as evidence about complexity classes.

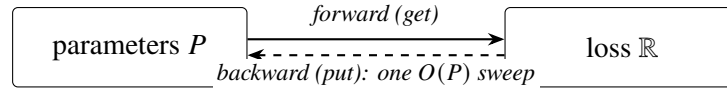
Here is the plan. Section 3 gives exact definitions, written and checked in the Lean proof assistant: what a witness is, what it means for the endpoints to determine it, and how much information it can hide. These definitions turn “the witness helps” from a hunch into a quantity each experiment can measure. Part I is the null result: on Mathlib code edits, explicit diff supervision adds nothing once a metric error is repaired. Part II is the hardware test: four machine-certified ZX circuit pairs on IBM `ibm_fez` show a real, replicated fidelity gain, with the noise model’s predicted ordering wrong and the mechanism shown to be duration, not gate count. Part III is the positive result: on Lean tactic prediction, where measured gold-tactic recoverability is 0.003, trace supervision wins by a large, content-dominated margin across three seeds. Part IV is the negative result: it tests the natural candidate law, that the gap grows with the witness fiber, and exact computation overturns it. The law is natural because more witnesses per endpoint pair means more hidden information; it still fails, because the evaluation’s own ceilings cap any gap, and the ceilings are computable before any training runs. Part V is the mechanism test: across a $32\times$ range of stage-2 budgets, the gap remains.

The paper has a second contribution: method. I pre-registered every experiment in a public repository before running it: twelve protocols, each with clear rules for what would prove me wrong, each verdict time-stamped before interpretation. Two registered designs ran, failed their own sanity checks, and are reported as failures. One script, `verify.py`, recomputes every number in every table from the committed files¹.

2 Background

Certificates. A decision problem is in NP when a “yes” answer comes with a witness that is fast to check. The practical point for this paper: finding a witness can be slow even when checking it is cheap. I use this only to choose test domains. The paper proves no complexity result.

Lenses: backpropagation works because the record is kept. Category theory describes a supervised learner as a *lens*: a forward map paired with a backward update, composed so that gradients flow through composites functorially^{2,3}. The backward pass exists because the forward pass kept its record. One backward sweep then costs what $2P$ forward sweeps would cost without it (Figure 1). Section 3 proves the seven laws of lenses in Lean; they are what let a deep stack of layers act as one lens.



finite differences: $2P$ forward passes; the lens keeps the record instead

Figure 1: A learner as a lens. The forward pass is the endpoints; the backward pass exists because the record of the forward sweep was kept. *Source: figure created by the author.*

ZX calculus: circuits as diagrams, rewriting as compilation. The ZX calculus turns quantum circuits into diagrams. The nodes are called spiders; each is a small linear map with a phase. The edges are wires, and each wire contracts tensor indices. Two families of rewrite rules, spider fusion and the bialgebra (Frobenius) equations, change a diagram’s shape without changing its meaning⁴. Kissinger and van de Wetering built a compiler on this idea: reduce the diagram fully, then read a circuit back out. It removes many of the expensive non-Clifford (T) gates, and it ships as PyZX’s `full_reduce`^{5,6}. The test circuits come from the T-par and optimization literature^{7,8}. Part II will ask hardware to trust this pipeline’s output, so I first reran the pipeline end to end. On the standard 30-circuit suite it cut the aggregate T-count by 46% (6019 to 3253), within 3% of the shipped T-par outputs head to head (2753 vs. 2829: five wins, eighteen ties, one loss), with a machine-checked equality witness for every circuit small enough to verify. That rerun is a reproduction with certificates, not a discovery, and the certificates are exactly what Part II needs: every optimized circuit it ships must be provably the same computation as its baseline. Table 1 gives representative rows; Figure 2 shows the pipeline on one pair.

Table 1: Representative rows from the 30-circuit benchmark rerun (T-counts; tpar = published T-par outputs). Extraction after full reduction inflates two-qubit counts, so hardware runs use phase teleportation: same 46% T-count win, two-qubit structure preserved, all witnesses green. *Source: project data*¹.

circuit	qubits	T in	T out	T (T-par)	verified
mod5_4	5	28	8	16	PASS
mod_mult_55	9	49	35	37	PASS
tof_3	5	21	15	15	PASS
barenco_tof_3	5	28	16	16	PASS
adder_8	24	399	173	215	n/a
gf2 ⁴ _mult	12	112	68	68	n/a

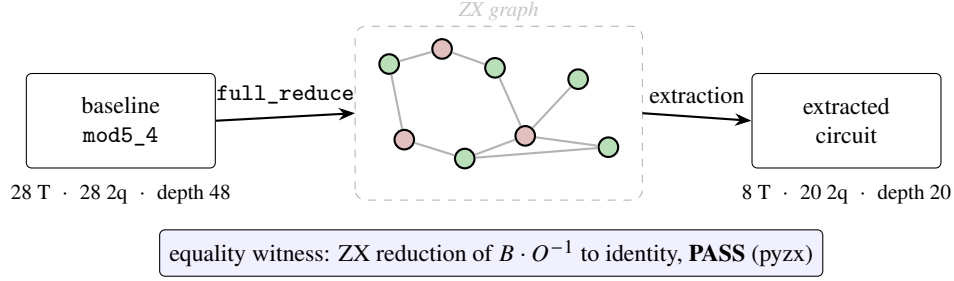


Figure 2: Circuit $\rightarrow$ graph $\rightarrow$ smaller circuit, on the mod5_4 pair. Nodes are spiders (linear maps with phases), edges are wires; rewriting fuses connected spiders of the same color. The layout is schematic, the counts are not: they are the certified pre-transpile numbers of Part II. *Source: figure created by the author from project data¹.*

Process versus outcome supervision. The question of this paper is old in machine learning: is it better to train on steps or on outcomes? Lightman et al. found that step-level (“process”) supervision clearly beat outcome supervision on competition mathematics, reaching 78% on a MATH subset⁹; Uesato et al. reported related findings with both paradigms¹⁰. On the theory side, Jia, Rakhlin, and Xie prove that under standard coverage assumptions, outcome data is statistically no harder to learn from than step data, up to polynomial factors in the horizon¹¹. Any real gap is therefore algorithmic, not an intrinsic statistical barrier. Parts I and III rediscover a known effect at small scale, and say so. The intended contribution is the controls around it: the metric repair of Part I, the shuffle and multi-seed controls of Part III, the mechanism test of Part V, and the negative result of Part IV.

Mirror circuits: the device checks itself. A mirror circuit runs a circuit followed by its inverse, and checks whether the input comes back. Proctor and collaborators developed the idea as a scalable benchmark of quantum computer capability¹². This project uses them as an application, not an invention: mirrors composed of a baseline circuit and its optimized partner make the hardware itself check the equality witnesses, with deranged (mismatched-pair) mirrors as a control against accidental returns (Section 5).

Lenses, ZX diagrams, and mirror circuits are the vocabulary this paper needs. The next section puts the central distinction itself into formal form.

3 The Formal Layer

When does keeping the record pay? To test the question, I first make it exact. This section defines the record, defines what it means for the endpoints to determine it, and says what the machine-checked proofs do and do not do for the paper.

3.1 Categorical preliminaries: the record as a morphism

Start with states and the operations that change them. Fix a set S of states and an alphabet Op of operations acting by $\text{act}: \text{Op} \times S \rightarrow S$. A *witness* of length L is a word $w = (o_1, \dots, o_L) \in \text{Op}^L$; write $\text{run}(w, s)$ for the iterated action. The witnessed transformations form a category $\mathbf{W}_L$: objects are states, a morphism $s \rightarrow t$ is a word w with $\text{run}(w, s) = t$, composition is concatenation, which is legal because $\text{run}(w_1 w_2, s) =$

$\text{run}(w_2, \text{run}(w_1, s))$, and the identity is the empty word. Forgetting the word yields the *endpoint category* $\mathbf{E}_L$ (same objects; a unique morphism $s \rightarrow t$ whenever some length- L word runs s to t) and a forgetful functor $U: \mathbf{W}_L \rightarrow \mathbf{E}_L$, the identity on objects and $w \mapsto (s, t)$ on morphisms.

Definition 1 (Recoverability and the fiber). *The system is recoverable at (s, L) when U is faithful on morphisms out of s : any two length- L words with the same action on s are equal. The fiber over an endpoint pair is the count $\varphi(s, t) = |U^{-1}(s \rightarrow t)|$ of length- L words running s to t . Recoverability is $\varphi \leq 1$ everywhere; otherwise $\log_2 \varphi$ measures, in bits, the witness information the endpoints cannot supply.*

Now read this as a statement about training. When $\varphi = 1$, the witness is a deterministic function of the endpoints, so any training signal it carries is already carried implicitly by the endpoint pair. Explicit witness supervision can then add nothing. Part I’s diffs are of this kind. When $\varphi > 1$ and inversion is search, the witness carries information no endpoint pair supplies. Part III’s tactics are of this kind. Part IV promotes φ to a candidate predictor and refutes it on its own terms.

Two further structures support the gradient and hardware sides. *Lenses*. A lens $(A, A') \rightleftarrows (B, B')$ is a pair $f: A \rightarrow B$ (forward) and $f': A \times B' \rightarrow A'$ (backward); composition with (g, g') has forward $g \circ f$ and backward $(a, c') \mapsto f'(a, g'(f(a), c'))$, so the backward pass of a composite decomposes through recorded intermediate values. This is exactly reverse-mode differentiation. *Frobenius algebras*. These are the algebraic rules behind ZX rewriting. A commutative Frobenius algebra $(A, \mu, \eta, \delta, \varepsilon)$ carries multiplication $\mu: A \otimes A \rightarrow A$ and comultiplication $\delta: A \rightarrow A \otimes A$ with (co)units, satisfying associativity and the Frobenius identity

$$(\text{id} \otimes \mu) \circ (\delta \otimes \text{id}) = \delta \circ \mu = (\mu \otimes \text{id}) \circ (\text{id} \otimes \delta). \quad (1)$$

In the ZX calculus the two spider colors are two such algebras, fused by a bialgebra law; spider fusion is associativity, and every rewrite the compiler of Part II applies is one of these equations. The companion Lean development formalizes the cobordism presentation Cob_2 and its symmetric monoidal quotient, and proves the connected-spider composition law

$$\text{spider}(a, b, g) \circ \text{spider}(b, c, h) = \text{spider}(a, c, g + (b - 1) + h), \quad (2)$$

where b is the shared boundary count and g, h the genera¹³. This is the equation that turns diagrammatic simplification from a heuristic into a certified compiler pass. With the objects in place, the rest of the section says what has been proved about them.

3.2 What is machine-checked, and where

Two Lean files and one companion repository carry the proofs. `LensLean.lean` (Mathlib-free) proves the seven category and monoidal laws for lenses: two identity laws, associativity, and four coherence equations. Every one is proved by `rfl`, which means the two sides are *definitionally* equal: the categorical structure was already present in the definitions, not imposed on top. The laws matter operationally, because they license fusing a deep composite of lenses into one backward sweep. A companion benchmark (`bench_lens.lean`)

measures what the license buys: at 1024 components, one fused sweep against central finite differences separates by a factor of 10^6 , and the lens gradient matches the analytic gradient exactly (Table 2).

Table 2: One reverse-mode lens sweep vs. central finite differences, timed under the Lean interpreter on a degree- P Horner polynomial (shapes meaningful, absolute values interpreter-bound). *Source: project data¹.*

P	lens (ms)	FD (ms)	FD / lens	lens vs. analytic	FD vs. analytic
8	0.00045	0.068	154	0.0	0.0
64	0.00046	1.88	4,110	0.0	4×10^{-6}
256	0.00122	34.5	28,224	0.0	3×10^{-6}
1024	0.00466	4,698	1,007,930	0.0	3×10^{-6}

`WitnessRecoverability.lean` formalizes Section 3.1’s axis. `free_faithful` proves one end case: in the free system (states are stacks, operations push), the endpoint literally contains the witness, so U is faithful at every length. Recovering the witness is reading. `abelian_not_faithful` proves the other end case: for two commuting bit flips, the two orderings are distinct witnesses with identical endpoints. The proof runs by kernel computation: `decide` evaluates the counterexample mechanically, with zero axioms. The counterexample is not argued; the kernel evaluates it. Part IV’s conditions interpolate between these two cases by mixing free and commuting generators, and `fiber.py` computes φ for them by exhaustive enumeration, matching the Lean `#eval` outputs on small systems. The companion repository holds the Frobenius side: the Cob_2 presentation, its quotients, and (2), all machine-checked against Mathlib v4.28.0¹³.

Table 3: The paper’s informal notions, their formal objects, and where each is machine-checked.

informal notion	formal object	Lean file
record of a computation	morphism of $\mathbf{W}_L$	<code>WitnessRecoverability.lean</code>
“the record adds nothing”	U faithful ($\varphi = 1$)	<code>free_faithful</code>
“the record carries bits”	fiber $\varphi > 1$	<code>abelian_not_faithful</code> (by <code>decide</code>)
compiler rewrite rule	Frobenius / spider equation	<code>Cob2*.lean</code> ¹³
gradient witness	lens composition laws	<code>LensLean.lean</code>

3.3 Axiom audit and provenance

An audit with `#print axioms` finds only `propext` and `Quot.sound` anywhere in the formal layer, Lean’s standard logical infrastructure. There are no sorry placeholders and no custom axioms. Every theorem kernel-checks in seconds.

Provenance disclosure, stated here rather than buried: Claude (Anthropic) drafted the recoverability file and the ceiling derivation of Part IV inside my categorical framework. I validated every definition, extended the file with the fiber layer and the demo computations, and I have rederived both proofs by hand. Lean proof search elsewhere in the broader program used Aristotle (Harmonic). I made every decision about what to formalize and every verdict call about what the formalization settles.

3.4 What work the Lean actually does

Two concrete answers, both used later. First, the formal layer *defines the axis*: the fiber of Section 7 is not a heuristic score but Definition 1’s quantity, computed exactly. Second, its exact consequences *stopped a registered experiment*: once the fiber is computable, its implications are computable, and the ceiling arithmetic of Section 7, run before the full sweep, showed the registered design could not see the effect it was built to measure. I stopped the sweep after its sanity check on the strength of that arithmetic. That is the work the Lean does: it made a conjecture precise enough to refute cheaply.

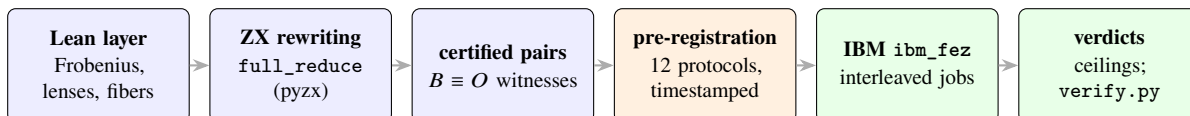


Figure 3: Provenance chain, redrawn from the repository. Every hardware number descends from a machine-checked equality witness and a pre-registered prediction; `verify.py` re-derives every table from committed artifacts. *Source: figure created by the author.*

4 Part I: Code Edits, Where the Record Is Cheap

4.1 Design

Start where recovery is easy. Given a file before and after an edit, any computer can work out the diff. So if the theory is right, training on the explicit diff should add nothing to training on the pair of files. The original protocol registered four predictions: P1, a diff-trained arm shows a zero-shot lead when asked to predict edits; P2, the headline, a durable advantage after a small shared adaptation; P3, a null on a control comparison; P4, protocol hygiene on both ends.

I mined 10,925 single-file edit records from the most recent 20,000 commits of the community `mathlib4` repository (shallow clone, 2026-07-18), sorted them by date, and held out the final 5%; evaluation scores the first 300 held-out items. The model is Pythia-410M with LoRA adapters (rank 16), bf16 on a GB10, seed 0¹⁴. Stage 1 trains three arms on identical data, budget (20M tokens each), and seed. Only the format differs: `ck_diff` sees BEFORE + DIFF (the record), `ck_after` sees BEFORE + AFTER (the endpoint pair), `ck_endpoint` sees AFTER alone. Stage 2 adapts each checkpoint on the same small diff-format dose (1M tokens), giving `ad_diff`, `ad_after`, `ad_endpoint`. The question: which stage-1 supervision best supports the shared downstream task of predicting edits?

4.2 The metric error, first-class

The first run (v1) seemed to confirm the headline. A skeptical second read took it apart, and that correction matters as much as anything that survived. The v1 score, mean character similarity to the gold diff, rewards copying the before-window, because a diff mostly repeats its context. Every adapted arm scored at the copying ceiling (≈ 0.243), so they looked equal on actual edit content. Worse, v1 saved only means, not per-item scores, so the conflation could not be audited after the fact. The repair is the *echo ceiling*: for each item, score

the similarity of the before-window itself to the gold diff. That is what pure copying earns. The headline metric is then $\text{sim} - \text{echo}$, the per-item gain over copying. Copying scores zero by construction. The repair removed the original claim. Pre-registration plus the audit habit caught the error, and I report both the failure and the fix rather than editing them away.

4.3 Results

The v2 suite scores every checkpoint per item on six measures: raw similarity (the v1 metric, kept for continuity), edit-line-only similarity, the echo ceiling, $\text{sim} - \text{echo}$, applicability (does the generation’s implied pre-image occur contiguously in the before-window), and implied-after similarity, which resists echo. Table 4 gives the full suite; Table 5 gives the surviving paired comparisons, recomputed independently by `verify.py` from the committed per-item scores (paired Wilcoxon signed-rank, $n = 300$).

Table 4: Part I full metric suite, all six checkpoints (seed 0, $n = 300$). The echo ceiling is identical across arms by construction; every arm scores below it on raw similarity, which confirms the v1 error a second time. *Source: project data¹.*

arm	sim (v1)	edit-line	echo	$\text{sim} - \text{echo}$	applies	implied-after	loss
ck_diff	0.2311	0.1634	0.2636	−0.033	7.7%	0.217	1.488
ck_after	0.1455	0.0036	0.2636	−0.118	6.3%	0.169	1.449
ck_endpoint	0.0887	0.0069	0.2636	−0.175	0.7%	0.065	1.444
ad_diff	0.2184	0.1645	0.2636	−0.045	9.7%	0.234	1.485
ad_after	0.2046	0.1428	0.2636	−0.059	8.0%	0.225	1.442
ad_endpoint	0.1706	0.1202	0.2636	−0.093	5.3%	0.099	1.443

Table 5: Part I surviving results (v2 metrics, seed 0, $n = 300$). The headline comparison is a null; both paired-format arms beat the endpoint-only arm decisively. *Source: project data¹.*

comparison	metric	mean Δ	wins to losses	p
ad_diff – ad_after	$\text{sim} - \text{echo}$	+0.014	159 to 140	0.152
ad_diff – ad_endpoint	$\text{sim} - \text{echo}$	+0.048	187 to 113	1.6×10^{-5}
ad_after – ad_endpoint	$\text{sim} - \text{echo}$	+0.034	184 to 116	2.4×10^{-4}

The bootstrap 95% interval for the headline is $[-0.006, +0.034]$, which includes zero. Table 6 gives the full paired analysis across all artifact-proof metrics. The pattern is consistent: on every metric that survives the echo repair, the two paired-format arms are statistically indistinguishable from each other and both sit clearly above the endpoint-only arm. For edit-line similarity the diff arm holds a marginal lead ($p = 0.063$, interval $[+0.002, +0.042]$), the one thread a seed-1 run would settle.

Two secondary observations sharpen the picture. Zero-shot, the record arm’s lead is real edit content: `ck_diff` emits at least one edit line on 95% of items, against 8% and 18% for the pair and endpoint arms, replicating at full n the five-sample autopsy that flagged the effect in v1. After adaptation, the endpoint arm’s behavior is diagnostic: it over-produces edit lines (mean 18.2 per item) with a 5% applicable rate. That is format without content.

Table 6: Part I complete paired analysis (Wilcoxon signed-rank, $n = 300$, seed 0). McNemar used for the binary applies metric. *Source: project data*¹.

comparison	metric	mean Δ	wins to losses	p
ad_diff – ad_after	sim – echo	+0.014	159 to 140	0.152
	edit-line sim	+0.022	159 to 136	0.063
	implied-after	+0.009	151 to 148	0.327
	applies (McNemar)		25 to 20	0.552
ad_diff – ad_endpoint	sim – echo	+0.048	187 to 113	< 0.001
	edit-line sim	+0.044	185 to 113	< 0.001
	implied-after	+0.135	242 to 58	< 0.001
ad_after – ad_endpoint	sim – echo	+0.034	184 to 116	< 0.001
	edit-line sim	+0.023	160 to 135	0.030
	implied-after	+0.126	237 to 61	< 0.001

4.4 Verdict

P1 confirmed again: the zero-shot lead is genuine edit content, not echo. P2, strong form (explicit diff format beats paired endpoints): not supported; after identical adaptation the two arms are indistinguishable. P2, weak form (paired path structure beats endpoint-only): strongly supported, on every artifact-proof metric. The plain statement: pairing transfers; serialization does not. What matters is keeping both states of the change in view, not writing the change as a diff. This is exactly what recoverability predicts: the diff is computable from the endpoints, so writing it out adds nothing. In the program’s framing: keep the section, not its integral. The section turns out to be the (before, after) pair itself, and the explicit diff is one way of writing it down, not a privileged form.

One note on independence: v1 ran remotely (fp32, T4); this run rebuilt the full pipeline locally (bf16, GB10) with one day of commit drift in the data window. It is an independent draw, not a re-measurement of v1’s checkpoints, and it reached the same verdict on the repaired metric.

Limitations carried forward. Single training seed; 410M scale; one-day commit drift against the v1 window; results specific to Mathlib-style single-file edits.

5 Part II: Quantum Circuits on Hardware

5.1 Design

Part II moves the test to a chip. I took four circuit pairs (mod5_4, tof_3, barenco_tof_3, mod_mult_55) from the Amy–Maslov–Nam benchmark family^{7,8}. Each pair implements one unitary twice: the standard baseline and a ZX-rewritten optimized circuit. ZX reduction machine-verifies pair equality before shipping (all four PASS), and `verify.py` re-verifies it independently from the committed QASM (Table 7). The record here is the compilation history: the rewrite proof that certifies the optimized circuit *is* the same computation. I registered the predictions before any hardware ran, under a 1% two-qubit depolarizing model, in size order: mod5_4 +0.048 > mod_mult_55 +0.029 > barenco_tof_3 +0.020 > tof_3 +0.013 (near-parity control). Execution as registered: both arms interleaved in one job per pair, 8192 shots per arm, optimization level 1, seed_transpiler=7, IBM ibm_fez, 2026-07-18¹⁵. A noiseless simulator check ran first: both

arms at fidelity 1.0, delta zero, so any gap on the chip is hardware, not a compile error. One note on the metric: all four circuits have deterministic ideal outputs, a single bitstring, so “fidelity” here means the probability of measuring the exact correct answer. The layout pass placed all four pairs on overlapping low-error subsets in the qubit 117 to 145 region, with mean CZ error 1.85 to 2.11×10^{-3} on the edges actually used, below the device median that day.

Table 7: Certified pairs, pre-transpile counts (baseline $\rightarrow$ optimized). Verification: ZX reduction of the composed pair to the identity. *Source: project data¹.*

circuit	qubits	2q gates	T-count	depth	verified
mod5_4	5	28 $\rightarrow$ 20	28 $\rightarrow$ 8	48 $\rightarrow$ 20	PASS
tof_3	5	18 $\rightarrow$ 16	21 $\rightarrow$ 15	31 $\rightarrow$ 30	PASS
barenco_tof_3	5	24 $\rightarrow$ 22	28 $\rightarrow$ 16	42 $\rightarrow$ 43	PASS
mod_mult_55	9	48 $\rightarrow$ 42	49 $\rightarrow$ 35	51 $\rightarrow$ 46	PASS

5.2 Results

Table 8: Part II main results, IBM `ibm_fez`, 2026-07-18. z from per-shot paired statistics; transpiled counts are post-layout. *Source: project data¹.*

pair	Δ observed	z	Δ predicted	transpiled 2q (b $\rightarrow$ o)	depth (b $\rightarrow$ o)
mod_mult_55	+0.0670	+9.5	+0.029	99 $\rightarrow$ 117	217 $\rightarrow$ 204
mod5_4	+0.0248	+4.8	+0.048	52 $\rightarrow$ 38	206 $\rightarrow$ 95
barenco_tof_3	+0.0167	+3.9	+0.020	42 $\rightarrow$ 34	164 $\rightarrow$ 126
tof_3 (control)	-0.0042	-1.1	+0.013	30 $\rightarrow$ 28	118 $\rightarrow$ 112

Two verdicts, per the registered decision tree. *Direction: confirmed* on all three effect pairs. The control landed at -0.004 , within shot noise of zero, which rules out a systematic interleaved-arm bias. *Ordering: falsified*, and stated as falsified. The registered ladder put `mod5_4` first and `mod_mult_55` second; observation inverted them. The 9-qubit circuit won by the largest margin despite carrying *more* transpiled two-qubit gates than its baseline (117 vs. 99). It won through depth (204 vs. 217): on a real device, coherence time prices circuits alongside gate error, so a one-parameter depolarizing model cannot rank them. Compilation pays on hardware; the simulator’s ordering does not transfer.

5.3 Error budget against the device ceiling

After the verdict (not pre-registered), I compared each arm against its ceiling from that day’s recovered calibration: $\prod(1 - \varepsilon_{CZ})$ over the exact edges used, times $\prod(1 - \varepsilon_{T0})$ over the exact qubits measured (Table 9). Seven of eight arms sit at 99% to 104% of ceiling: the runs saturate the unmitigated hardware limit, so the measured deltas are close to a pure compilation effect. The one sub-ceiling arm is the `mod_mult_55` baseline at 89%. It carries about eight points of error the gate-count model cannot explain, while its optimized arm

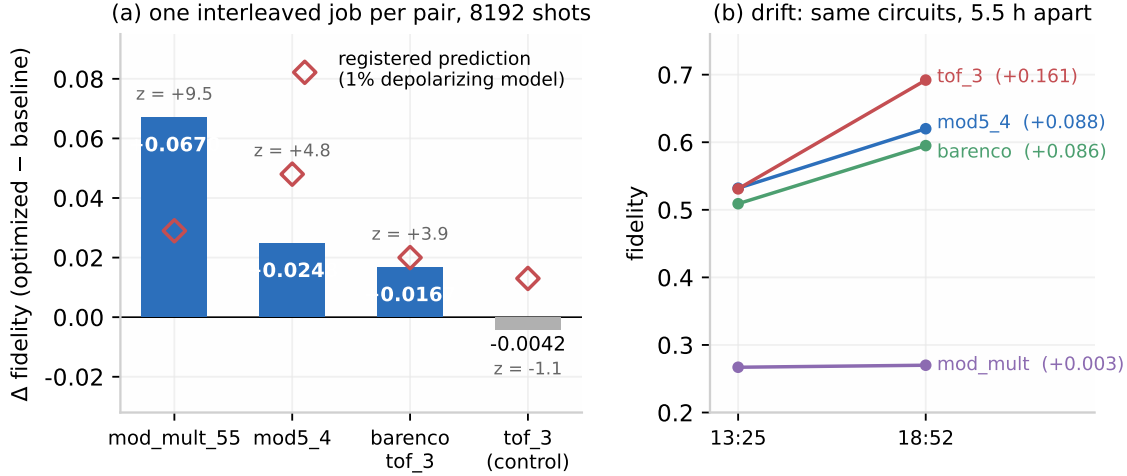


Figure 4: Part II results in two panels. (a) Observed deltas against the registered predictions, one interleaved job per pair, 8192 shots, IBM `ibm_fez`, 2026-07-18. (b) The drift measurement behind the interleaving rule: identical circuits in identical no-DD condition, re-measured hours later, move by up to +0.161. *Source: figure created by the author from project data¹.*

on mostly the same qubits sits at 103%. The excess tracks circuit duration, which independently supports the duration reading of the ordering anomaly. Values slightly above ceiling are physical, not anomalous: randomized benchmarking counts all Pauli errors¹⁶, but for circuits whose output is a computational-basis state, Z-type errors near measurement are invisible. The RB-derived ceiling is therefore mildly conservative for this family.

Table 9: Error budget vs. execution-time calibration (recovered from the four jobs' backend properties). Above-ceiling values are physical for this circuit family (see text). *Source: project data¹.*

pair	arm	ceiling	measured	% of ceiling
mod5_4	baseline	0.875	0.865	99%
mod5_4	optimized	0.902	0.890	99%
tof_3	baseline	0.908	0.941	104%
tof_3	optimized	0.913	0.937	103%
barenco_tof_3	baseline	0.892	0.912	102%
barenco_tof_3	optimized	0.907	0.928	102%
mod_mult_55	baseline	0.758	0.678	89%
mod_mult_55	optimized	0.726	0.745	103%

One further covariate qualifies the registered number. A transpile-seed sweep (20 seeds, level 1, Fez snapshot) shows seed 7 gave the `mod_mult_55` optimized arm its *worst* routing of all twenty (117 2q; sweep minimum 102, median 112.5), while the baseline drew near-best routing (99; minimum 96). The registered +0.067 was therefore measured with the optimized arm at its worst draw, and is plausibly a lower bound on the compilation effect at this site.

5.4 Replication, routing decomposition, and drift

Six days later I re-ran the flagship pair under stricter control (R2'): both arms pinned to one shared nine-qubit layout, one interleaved job. Baseline 99 2q at fidelity 0.639; optimized 102 2q at 0.700. The delta, +0.0604 against the registered +0.0670, shows the compilation gain replicates at matched placement. A third arm on identical qubits with deliberately worse routing (114 2q) measured 0.565: a routing effect of -0.1346 for 12 extra two-qubit gates and 18 extra depth. With that day's calibration (mean CZ 5.38×10^{-3} vs. 4.63×10^{-3} on the good edges), the ceiling model explains 0.077 of the 0.135 drop: 57%, the corrected figure, superseding an earlier informal $5\times$ estimate that used the wrong CZ error. The remaining ≈ 0.058 tracks depth. Routing skill matters, but depth has its own measurable price.

The device state that day is part of the record: CZ errors on these qubits ran 4.48 to 5.38×10^{-3} , more than double the 2.11×10^{-3} measured on 2026-07-18. All three arms measured above their calibration ceilings (107%, 121%, 112%), consistent with the known conservatism of RB-derived ceilings for computational-basis outputs.

Why interleave at all? Because the chip drifts. Same circuits, same no-DD condition, two jobs 5.5 hours apart: `mod5_4` 0.532 $\rightarrow$ 0.620; `tof_3` 0.531 $\rightarrow$ 0.692; `barenco` 0.509 $\rightarrow$ 0.595; `mod_mult` 0.267 $\rightarrow$ 0.270. That is up to +0.161 in an afternoon, and +0.03 to +0.08 in the 32 minutes between two consecutive jobs. These swings exceed most effects measured in this project, including the registered +0.067. Any cross-job comparison in this work is therefore uninterpretable, which is why the cross-job DD comparison (R1) is invalid as executed.

5.5 The mirror protocol, dynamical decoupling, and two rejected explanations

The mirror experiment makes the hardware itself check the equality witnesses. For each registered pair p , a *true mirror* runs $X(x) \cdot B_p \cdot O_p^{-1}$ on a random input basis state $|x\rangle$ (seeded, recorded), with barriers so the transpiler cannot cancel the halves; if $B_p \equiv O_p$ the composition returns exactly x . A *deranged mirror* runs $X(x) \cdot B_p \cdot O_q^{-1}$ with $q \neq p$, and x is redrawn until the ideal output differs from x , which guards against inputs that happen to be fixed points of the composition. The metric is the return probability $r = P(\text{measure exactly } x)$ at 8192 shots, all circuits in one interleaved job. Three predictions were registered: M1, every true mirror returns $r \geq 0.25$ with the ordered contrast $r(\text{true}_p) > r(\text{deranged}_p) + 0.3$; M2, point predictions from the product of the per-arm fidelities (tolerance ± 0.15); M3, every deranged mirror stays at $r \leq 0.15$. One honest design note belongs in the record: the first draft placed a random Pauli layer at the joint, which is wrong for non-Clifford circuits, because the layer conjugates through B and the expected string then requires simulation to compute, defeating the protocol. Randomized inputs give the same protection against coherent cancellation with a simulation-free expectation. The correction was logged before submission.

Results. True mirrors returned 0.267 to 0.692; deranged mirrors 0.0002 to 0.086. M3 confirmed; the derangement control works. The M2 point predictions failed low on all four pairs. Two hypotheses were registered to explain the undershoot. The idle-dephasing hypothesis predicted that suppressing dephasing during idle time would raise true-mirror returns. The metric-blindness hypothesis held that per-arm fidelity, scored on computational-basis histograms, is blind to phase errors that a mirror exposes through O^{-1} .

The first test of idle dephasing (R1) compared a DD job at 13:25 against a no-DD job at 18:20: five hours of device drift confounded with the treatment, in direct violation of this project’s own interleaving protocol. Its measured deltas were mixed in sign (mod5_4 -0.033 , tof_3 $+0.089$, barenc0 $+0.055$, mod_mult_55 -0.115 ; mean -0.001 ; deranged max 0.086), so R1a/b failed and R1c confirmed, and the verdicts stand as recorded. But the run does not isolate a DD effect; two of its four sign readings were later shown to be drift.

The corrected rerun (R1’) put all sixteen circuits, eight mirrors times {no-DD, DD}, in one job, with DD as an XX sequence of native X pulses. Result: uniformly harmful. Paired deltas -0.076 , -0.022 , -0.005 , -0.039 , mean -0.036 . Deranged mirrors stayed under 0.15 as registered (max 0.070), so DD did not manufacture signal. The added pulses, 80 to 225 per circuit, cost more error than the idle dephasing they suppress at this scale. Per the falsification clause, the idle-dephasing explanation is **rejected**. The metric-blindness hypothesis remains **untested** by any run so far, and I label it as such rather than carry it quietly. One data-integrity caveat from this round: the ceilings recorded for the DD arms are artifacts, because DD pads idle qubits across the full register, so the qubit-extraction step counts all 156 qubits and the readout product collapses the ceiling to 0.039 . The no-DD ceilings are valid; the DD-arm ceilings are not used anywhere.

5.6 Two design errors, owned

Cross-job DD (R1). The first DD run compared a DD job at 13:25 against a no-DD job at 18:20: five hours of device drift confounded with the treatment, in direct violation of this project’s own interleaving protocol. Its recorded verdicts stand as measured, but the run does not isolate a DD effect; two of its four sign readings were later shown to be drift. R1’ is the corrected measurement.

Seed changes layout (R2). The seed-11 rerun treated `seed_transpiler` as a routing knob. It is not: the seed changes the layout, and the two arms landed on disjoint physical qubits (0/9 overlap, versus 7/9 in the registered runs). Its $+0.171$ delta therefore compared two different qubit sets and is not comparable to the registered $+0.067$. R2’ is the corrected measurement, and the seed sweep that exposed the error is committed alongside it.

6 Part III: Proof Steps, Where Recovery Is Search

6.1 From Part I to Part III

Part I’s null says explicit record supervision adds nothing where the record is determined by the endpoints. The measured diagnostic for the other end: in the proof-step domain, the fraction of gold tactics appearing anywhere as a substring of the concatenated input states is 0.003 . Diffs in Part I are fully determined by their endpoints; tactics are, for practical purposes, absent from them. Checking a tactic is cheap, since Lean’s kernel does it, but reconstructing one from the goal states is search. If recoverability is the variable, this is where trace supervision should pay.

6.2 Design

Data: LeanDojo benchmark 4, mathlib4 tactic steps, split by theorem name so no held-out theorem leaks into training¹⁷. Model: Pythia-160M, fp32, seed 0 for the registered run and seeds 1 and 2 as replicates¹⁴. Three stage-1 corpora at matched token budget: `pw_trace` (state-before + tactic + state-after, the full record), `pw_pair` (the two endpoint states), `pw_endpoint` (the after-state alone). Stage 2 is an identical small fine-tune for every arm on the eval format: given both states, predict the tactic. Metrics: exact match, normalized similarity, and `sim - echo` as repaired in Part I. I registered three predictions. P1: `pw_trace` beats `pw_pair` on `sim - echo` (paired Wilcoxon $p < 0.05$, mean $\geq +0.02$), the registered *reversal* of Part I’s null. P2: `pw_pair` beats `pw_endpoint`. P3, the falsification branch: a `pw_trace` $\approx$ `pw_pair` tie would drop the complexity framing from the writeup, not soften it. A registered addendum added the format-exposure control: `pw_shuffle` trains on BEFORE + a real but *wrong* tactic (a derangement over training steps) + AFTER, so it sees tactic-format text with no valid content. C1: `pw_trace` beats `pw_shuffle`, meaning the gain is witness content. C2: `pw_shuffle` vs. `pw_pair`, reported as observed.

6.3 Results across three seeds

Table 10: Part III, multi-seed replication. The headline is the range, not a point estimate. *Source: project data¹.*

	seed 0	seed 1	seed 2
C1: trace – shuffle	+0.0393 ($p = 1.4 \times 10^{-5}$)	+0.0500 ($p = 3.3 \times 10^{-9}$)	+0.0240 ($p = 0.0055$)
C2: shuffle – pair	mean +0.0122, all $p < 0.05$		
P2: pair – endpoint	null in all seeds ($p = 0.60, 0.28, 0.77$)		

At the registered seed, P1 confirmed at +0.0540 (202 to 87, $p = 3.4 \times 10^{-13}$). The multi-seed round registered three checks before running: R3a, C1 stays positive with $p < 0.05$ in every seed; R3b, the across-seed standard deviation of the C1 mean delta stays below 0.02; R3c, P2 remains null in every seed. All three confirmed. C1 holds at +0.0393, +0.0500, +0.0240 across seeds 0, 1, 2 (sd 0.0131); C2 averages +0.0122 (sd 0.0046), significant in every seed; P2 stays null ($p = 0.60, 0.28, 0.77$). But the C1 magnitude ranges from +0.024 to +0.050, a factor of two. Sign and significance are robust; magnitude is not, so the range is the reported quantity. The decomposition is stable: content (C1) dominates format (C2), with content share 76% across seeds (73% at the Part III operating point, $N = 2000$ stage-2 examples).

6.4 Honest limitation, and a logged deviation

All arms score below the copy baseline in absolute terms: `sim - echo` is negative throughout, because at 160M parameters the model cannot actually do tactic prediction. Every comparison in this section is therefore a valid comparison *among models that cannot perform the task*. The effect is real, paired, replicated, and content-dominated, and it lives in a regime where the absolute capability is zero. Larger models are untested.

One protocol deviation is logged in the repository and belongs in the paper: the first registered training run diverged (loss spike, unrecovered). Per the registration’s deviation rule, I committed the fix (a lowered

stage-1 learning rate, 5×10^{-5} , unchanged across arms) and the rerun with dated notes before any evaluation ran. Every number reported here comes from the rerun.

6.5 The tautology objection

Isn't "training on tactics helps predict tactics" a tautology? Three registered controls stand between the claim and that objection. Stage 2 equalizes task exposure: every arm receives the identical fine-tune on the identical eval format, so all arms have seen what the task *is*. The shuffle arm equalizes format: it too saw tactic-shaped text in stage 1, and it loses to the trace arm by the full C1 margin. Seeing the format is not what transfers. What remains is content, and Part V shows volume does not equalize that either: $32\times$ more stage-2 examples do not close the gap.

7 Part IV: The Formalized, Falsified Predictor

Parts I and III found opposite results in two domains. The natural next guess: the size of the gap depends on the fiber, Definition 1's count. More witnesses per endpoint pair means more hidden information, so the gap should grow with the fiber. Part IV made that guess precise, registered it, and refuted it.

7.1 The registered design

Six conditions, each a system of eight operations on a 28-bit state with witness length $L = 6$. Vocabulary, witness length, state representation, output length, and operation names are identical across conditions; names are opaque (OP0...OP7) so no structure leaks through labels. Only the algebra differs, interpolating between the two end cases proved in Section 3. I computed fiber sizes *exactly*, by full enumeration of all $8^6 = 262,144$ witnesses per condition: readable 1.0, free 1.0, q25 2.8, q50 29.7, q75 349.1, abelian 2581.0. Four arms per condition (trace / pair / endpoint / shuffle) at matched character budget, trained under the Part III protocol (Pythia-160M, fp32, stage-1 lr 5×10^{-5} , stage-2 lr 2×10^{-5} , seed 0 registered, seeds 1 and 2 as replicates): 24 training runs in total. The eval asks the model to generate the witness from the endpoint pair; the primary metric is token accuracy over the six operation slots, chance = 0.125. The witness alphabet is disjoint from the state alphabet, so copying the input cannot beat chance and no echo correction is required. Registered predictions: P4a, the gap grows with $\log_2$ fiber (slope > 0 , $p < 0.05$, $r^2 \geq 0.5$); P4b, free exceeds readable by ≥ 0.05 (same fiber, opposite invertibility); P4c, the sanity anchor: on readable, where the end state literally contains the witness, all arms exceed 0.80 and the gap is below 0.05; P4d, content share above 50% in the high-fiber conditions.

7.2 The failed sanity check

P4c failed on the first condition. Every arm scored at chance, 0.121 to 0.161 against a chance level of 0.125, because the state was rendered as a binary string and the model could not decode nibbles. The registration said that if the sanity anchor fails, no other prediction in the file may be interpreted. I stopped the sweep after one condition, on the registered rule.

7.3 The computation that retired the design

Working through the failure exposed something deeper than the encoding. The gap must vanish at *both* ends of the fiber axis. At fiber 1, the inverse is learnable and stage 2 alone teaches both arms, so the low end fails. At large fiber, the witness is not determined by the endpoints, so *no* arm can exceed a near-chance ceiling, and the high end fails. Table 11 makes this exact: Bayes ceilings by full enumeration, and the maximum possible gap (ceiling minus chance) at each condition.

Table 11: Exact Bayes ceilings (generation-from-endpoints eval, $L = 6$). The maximum gap any supervision format could produce collapses at both ends of the fiber axis. *Source: project data¹.*

condition	mean fiber	Bayes ceiling	max gap
readable	1.0	1.000	0.875
free	1.0	1.000	0.875
q25	2.8	0.915	0.790
q50	29.7	0.739	0.614
q75	349.1	0.514	0.389
abelian	2581.0	0.190	0.065

At the abelian end the maximum achievable gap is 0.065, within noise of any realistic measurement. And at fiber 1, the free condition caps *both* arms computationally: the witness there is determined by the endpoints only through inverting six mixing maps, which the model cannot do either. The low end fails too, which also invalidated P4b as registered.

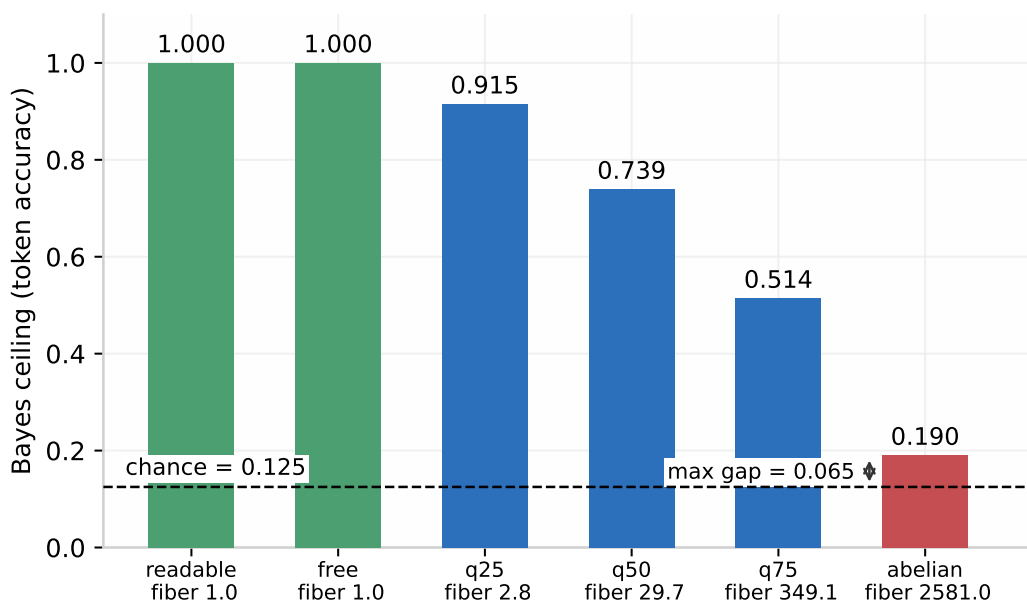


Figure 5: The impossibility figure. Exact Bayes ceilings per condition (fiber sizes in parentheses), chance level dashed. No monotone law from fiber size to the supervision gap can pass through these bars. *Source: figure created by the author from project data¹.*

The registration had also committed, as illustration rather than test, to placing Part I (diffs) and Part III

(tactics) on the fitted fiber curve after the fact, with estimated fibers and explicit uncertainty. With the curve retired, that placement is moot. The Part I versus Part III contrast stands as a recoverability-controlled domain difference, measured at the endpoints by the 0.003 diagnostic, rather than as evidence for a quantitative law.

7.4 The claim, exactly sized

For this family of operation systems, under generation-from-endpoints evaluation, *no monotone law from fiber size to the supervision gap can exist*: the achievable gap is bounded by ceilings that collapse at both ends of the axis. This is a claim about this family and this evaluation mode, not a general impossibility theorem. Its provenance is the point: the quantity was defined in Lean, its consequences computed exactly in `ceiling.py`, and the conjecture it refutes was pre-registered before the computation refuted it. The standalone lesson is the pair of `readable` and `free`: identical fiber (1.0), opposite computational invertibility. Information and computation are separate axes, and the formal quantity, fiber size alone, is incomplete by construction.

8 Part V: The Mechanism, Tested by Budget

8.1 Question and design

Part IV retired the information-theoretic explanation. Two mechanism candidates remain for the Part III advantage. *Data efficiency*: the pair arm never sees witness tokens in stage 1, so it must learn the witness vocabulary and distribution from stage 2 alone; enough stage-2 data should erase the difference. *Structural transfer*: stage-1 trace exposure conveys something about the state-to-tactic correspondence that endpoint data cannot supply at any stage-2 budget. The registered design holds stage 1 fixed per arm and varies the stage-2 budget over $N \in \{125, 250, 500, 1000, 2000, 4000\}$, with nested draws (each smaller set a prefix of the next), so that at a given (N, seed) every arm sees identical examples in identical order. Registered predictions: P5a, the gap decreases with $\log_2 N$ (slope < 0 , $p < 0.05$); P5b, at $N = 4000$ the gap is below $+0.02$ and no longer significant; P5c, the anchor: at $N = 2000$ the gap lands within ± 0.025 of Part III's $+0.054$.

Two known sources of slack were stated before running, so the anchor check is distributional, not exact: Part III drew its stage-2 set with `random.sample` while this design uses shuffle-then-prefix to make budgets nested, so the $N = 2000$ subset differs from Part III's subset of the same pool; and stage 1 runs as a separate process here, so the torch RNG state entering stage 2 differs, perturbing batch order. A small- N caveat was also stated: at $N = 125$ stage 2 is roughly four optimizer steps, so the smallest budgets carry high variance; all six points enter the regression regardless.

8.2 Results

Verdicts. P5c **passed**: $+0.0527$ against the $+0.054$ anchor, a miss of 0.0013 . The pipeline measures what Part III measured. P5a **failed**: slope -0.0057 on six points, $r^2 = 0.335$, $p = 0.229$, no significant decay. P5b

Table 12: Part V budget curve, seed 0. The pair arm plateaus from $N = 1000$; the trace arm keeps improving to its best at $N = 4000$. *Source: project data¹.*

N	pw_trace	pw_pair	gap	p
125	-0.0736	-0.1823	+0.1086	2.0×10^{-27}
250	-0.0699	-0.1483	+0.0784	3.0×10^{-15}
500	-0.0715	-0.1397	+0.0682	3.5×10^{-11}
1000	-0.0573	-0.1302	+0.0729	3.1×10^{-14}
2000	-0.0718	-0.1245	+0.0527	5.8×10^{-9}
4000	-0.0427	-0.1256	+0.0829	1.2×10^{-13}

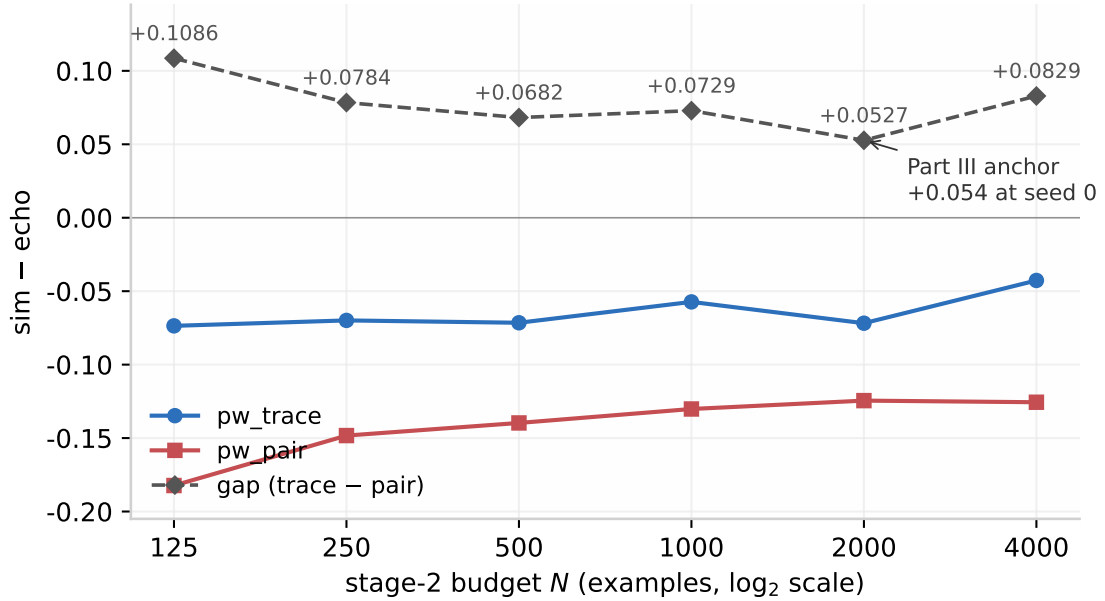


Figure 6: The budget curve: arm scores and the gap against the stage-2 budget on a $\log_2$ scale. The gap at $N = 2000$ anchors Part III; the gap at $N = 4000$ is the largest since $N = 125$. *Source: figure created by the author from project data¹.*

failed decisively: at $N = 4000$ the gap is $+0.0829$, $p = 1.2 \times 10^{-13}$, the largest since the four-optimizer-step regime at $N = 125$. Extrapolating the flat, insignificant slope anyway gives a zero-crossing near $2^{22.9} \approx 8.0$ million examples; I report it as an extrapolation beyond the tested range, which is what it is. The pattern shows in the arm scores, not just the gap: the pair arm plateaus at ≈ -0.125 from $N = 1000$ onward, while the trace arm keeps improving to its best score at $N = 4000$. The 4000-vs-2000 uptick ($+0.030$, $p = 0.079$) is suggestive only, and I label it as such.

8.3 The registered conditional: decomposition at $N = 4000$

Because P5b failed, the pre-registered conditional took effect: train `pw_shuffle` stage 1, fine-tune once at $N = 4000$, and decompose the surviving gap. Content (trace – shuffle): $+0.0563$ (176 to 99, $p = 5.2 \times 10^{-6}$). Format (shuffle – pair): $+0.0266$ (175 to 108, $p = 9.0 \times 10^{-5}$). Content share 68%, against Part III’s 73% at $N = 2000$ (76% across seeds). The composition of the gain survives the budget test, not only its size.

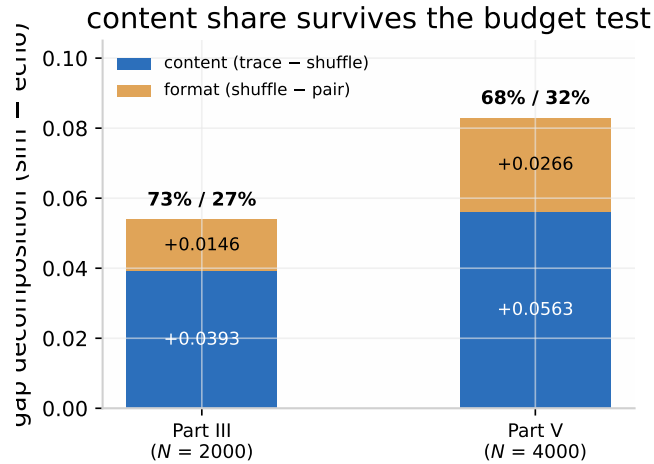


Figure 7: Content/format decomposition at the two operating points. The content share is stable (73% to 68%) across a doubling of the stage-2 budget. *Source: figure created by the author from project data¹.*

8.4 The final claim, exactly sized

The trace advantage in this domain is majority witness content, is not format familiarity, and is not purchasable with endpoint fine-tuning data across a $32\times$ budget range. Two uncertainty statements belong inside the claim, not in a footnote: the curve is single-seed, and Part III’s own cross-seed magnitude swing is a factor of two. The honest statement is that the gap persists everywhere it was measured, with magnitude known only to that factor.

Table 13: The whole result in one table. Recoverability, not fiber size, separates the domains; the mechanism evidence comes from the controls, not the headlines.

domain	witness recoverable?	supervision gap	mechanism evidence
code diffs	yes (determined)	null ($p = 0.152$)	n/a (echo-repaired)
proof steps	0.003 verbatim	+0.024 to +0.050	68 to 76% content; not buyable (Part V)

9 Synthesis

The honest general statement: where the record of a computation is recoverable from its endpoints, supervising on it adds nothing measurable. Where recovery is search, trace supervision transfers something that endpoint data does not supply at any budget tested, and most of what transfers is content, not format. The non-claims are part of the result, listed rather than implied: nothing here bears on P versus NP; the fiber is falsified as the predictor of when trace supervision pays, and the correct predictor is open; larger models are untested; the hardware result is one device, one calibration era, one registered seed.

10 Methods, Rigor, and Disclosure

Pre-registration ledger. I registered twelve protocols as timestamped commits to the project repository before the corresponding runs, and recorded verdicts against them before interpretation. Two registered designs (R1, R2) are reported as confounded alongside their corrected reruns.

Table 14: Pre-registration ledger. Short hashes refer to the project repository¹.

protocol	registered (UTC)	commit	verdict status
Part I: edit-witness P1 to P4	2026-07-18	89da3fc	complete; v1 metric error repaired
Part II: hardware ladder + control	2026-07-18	89da3fc	direction confirmed; ordering falsified
Part III: P1/P2/P3	2026-07-24	dc3dcaa	P1 confirmed; P2 falsified
Part III addendum: shuffle C1/C2	2026-07-24	930b2b2	C1 confirmed; C2 observed positive
Mirror M1 to M3 (hardware witness)	2026-07-24	59a1385	M3 confirmed; M2 failed low on all four
Round 2 R1 (DD mirror)	2026-07-24	671ac0d	confounded (cross-job); rerun as R1'
Round 2 R2 (seed-11 rerun)	2026-07-24	671ac0d	confounded (layout); rerun as R2'
Round 2 R3 (Part III multi-seed)	2026-07-24	671ac0d	R3a/b/c all confirmed
Round 3 R1' (drift-controlled DD)	2026-07-24	563edcd	DD uniformly harmful; dephasing rejected
Round 3 R2' (matched layout)	2026-07-24	563edcd	R2'a/b/c all confirmed
Part IV: fiber law P4a to P4d	2026-07-24	75f0e4c	P4c failed; design retired by exact ceilings
Part V: budget curve P5a to P5c + cond.	2026-07-25	ae895f7	P5c passed; P5a/b failed; conditional ran

Verification. `verify.py` (dependencies: `pyzx`, `qiskit`, `qiskit-aer`, `scipy`) runs 39 checks and re-derives every table in this paper from committed artifacts¹: independent pair-equality verification from the committed QASM, manifest counts, transpile identity against the recorded hardware runs, noiseless aer reproduction, hardware-delta significance, ceiling saturation against the recovered 2026-07-18 calibration, and the Part I statistics recomputed from committed per-item scores. The reader is invited to run it.

Reproduction. Every part reruns from the repository root with the committed scripts, smoke-tested on CPU first as registered:

- Part I: mine the diffs with `mine_mathlib_diffs.py`, train with `train_arms.py` (20M tokens, then 1M adaptation), evaluate with `eval_arms.py` at $n = 300$.
- Part II: `bench_zx.py` (classical suite), then the hardware runner, one interleaved job per pair.
- Part III: `prep_data.py`, `train.py`, `eval.py`, `stats.py`.
- Part IV: `fiber.py --L 6` (exact enumeration), `ceiling.py` (exact ceilings).
- Part V: `train_stage1.py` once per arm, then `train_stage2.py` and `eval_budget.py` at each budget N , then `stats_budget.py`.

Tool disclosure, specific. Lean proof search across the broader formal program used Aristotle (Harmonic). Claude (Anthropic) contributed engineering scaffolding, analysis drafting, and the initial drafts of the error-budget analysis, `verify.py`, the recoverability formalization, and the ceiling derivation. Every experimental design decision, every verdict call against the pre-registrations, and every sentence of prose in this paper is mine. I rederived by hand: the two faithfulness proofs, the ceiling bound, and the error-budget computation. Where this paper states a result, I can defend the derivation. The bias throughout is to underclaim rather than overclaim.

Data and code availability. All experiments, pre-registrations, raw JSON artifacts, and `verify.py` are public in the project repository¹. The formal mathematics behind the ZX rewrite rules, the Frobenius/cobordism presentation, its symmetric monoidal quotients, and the condensed target, is developed in the companion Lean repositories¹³: 12,777 lines across 33 files, no proof placeholders, no custom axioms, machine-checked against Mathlib v4.28.0^{18,19}. The classical benchmarking behind Figure 2’s pipeline, a 46% aggregate T-count reduction across the 30-circuit Amy–Maslov–Nam suite, within 3% of the shipped T-par outputs head to head, every verifiable instance witnessed, is reported in the repository’s `benchmarks_report.md` as a reproduction of⁵, not a new method.

11 Limitations and Future Work

One quantum device and one calibration era. One model scale, 160M parameters. A single-seed budget curve with a factor-of-two cross-seed magnitude uncertainty. The metric-blindness hypothesis for the mirror under-shoot remains untested, labeled rather than carried. Registered hardware follow-ups, in order of information per dollar: the transpile-seed sweep on `mod_mult_55` (done, Section 5); a second device on a different day, to test whether the ordering scrambles identically; optimization level 3, to test whether the vendor transpiler equalizes the arms, which is itself a finding; and the mirror protocol at larger package sizes. Two further directions, named directly. First, the verification-eval experiment: Parts I to V measure whether trace supervision helps *generation*; the open question is whether it helps *checking*, whether witness-trained models are better verifiers. The Jia–Rakhlin–Xie equivalence suggests this is the direction doing the work¹¹. Second,

the revived predictor: with the fiber falsified, the quantity to formalize next is a sample-complexity measure, the number of endpoint examples needed to match the trace arm. Part V’s curve makes this estimable in principle, and a future registration can test it directly.

References

- [1] Sean Mahdavian. Applied-experiments-of-cmcm: pre-registered witness-supervision and verified-zx experiments, with all raw artifacts, pre-registrations, and `verify.py`. <https://github.com/seanm27101/Applied-Experiments-of-CMCM>, 2026.
- [2] Brendan Fong, David I. Spivak, and Rémy Tuyéras. Backprop as functor: A compositional perspective on supervised learning. In *Proceedings of the 34th Annual ACM/IEEE Symposium on Logic in Computer Science (LICS)*, 2019.
- [3] Geoffrey S. H. Cruttwell, Bruno Gavranović, Neil Ghani, Paul Wilson, and Fabio Zanasi. Categorical foundations of gradient-based learning. In *Programming Languages and Systems (ESOP)*, 2022.
- [4] Bob Coecke and Ross Duncan. Interacting quantum observables: Categorical algebra and diagrammatics. *New Journal of Physics*, 13:043016, 2011.
- [5] Aleks Kissinger and John van de Wetering. Reducing the number of non-clifford gates in quantum circuits. *Physical Review A*, 102:022406, 2020.
- [6] Aleks Kissinger and John van de Wetering. Pyzx: Large scale automated diagrammatic reasoning. In *Proceedings 16th International Conference on Quantum Physics and Logic (QPL/ACT)*, 2019.
- [7] Matthew Amy, Dmitri Maslov, and Michele Mosca. Polynomial-time t-depth optimization of clifford+t circuits via matroid partitioning. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 33(10):1476–1489, 2014.
- [8] Yunseong Nam, Neil J. Ross, Yuan Su, Andrew M. Childs, and Dmitri Maslov. Automated optimization of large quantum circuits with continuous parameters. *npj Quantum Information*, 4:23, 2018.
- [9] Hunter Lightman, Vineet Kosaraju, Yuri Burda, Harrison Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let’s verify step by step. *arXiv:2305.20050*, 2023. ICLR 2024.
- [10] Jonathan Uesato, Nate Kushman, Ramana Kumar, Francis Song, Noah Siegel, Lisa Wang, Antonia Creswell, Geoffrey Irving, and Irina Higgins. Solving math word problems with process- and outcome-based feedback. In *Advances in Neural Information Processing Systems 35 (NeurIPS)*, 2022.
- [11] Zeyu Jia, Alexander Rakhlin, and Tengyang Xie. Do we need to verify step by step? rethinking process supervision from a theoretical perspective. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, 2025.
- [12] Timothy Proctor, Kenneth Rudinger, Kevin Young, Erik Nielsen, and Robin Blume-Kohout. Measuring the capabilities of quantum computers. *Nature Physics*, 18:75–79, 2022.
- [13] Sean Mahdavian. Liquid tqft in lean 4: machine-checked frobenius/cobordism infrastructure (canonical implementation and synchronized paper snapshot). <https://github.com/seanm27101/Liquid-TQFT-CMCM-cont>. and <https://github.com/seanm27101/Liquid-TQFT-lean-CMCM>, 2026.

- [14] Stella Biderman, Hailey Schoelkopf, Quentin Anthony, Herbie Bradley, Kyle O’Brien, Eric Hallahan, Mohammad Aflah Khan, Shivanshu Purohit, USVSN Sai Prashanth, Edward Raff, Aviya Skowron, Lintang Sutawika, and Oskar van der Wal. Pythia: A suite for analyzing large language models across training and scaling. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, 2023.
- [15] Ali Javadi-Abhari, Matthew Treinish, Kevin Krsulich, Christopher J. Wood, Jake Lishman, Julien Gacon, Simon Martiel, Paul D. Nation, Lev S. Bishop, Andrew W. Cross, Blake R. Johnson, and Jay M. Gambetta. Quantum computing with qiskit. *arXiv:2405.08810*, 2024.
- [16] Easwar Magesan, Jay M. Gambetta, and Joseph Emerson. Characterizing quantum gates via randomized benchmarking. *Physical Review A*, 85:042311, 2012.
- [17] Kaiyu Yang, Aidan Swope, Alex Gu, Rahul Chalamala, Peiyang Song, Shixing Yu, Saad Godil, Ryan Prenger, and Anima Anandkumar. Leandojo: Theorem proving with retrieval-augmented language models. In *Advances in Neural Information Processing Systems 36 (NeurIPS), Datasets and Benchmarks*, 2023.
- [18] The mathlib Community. The lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP)*, 2020.
- [19] Leonardo de Moura and Sebastian Ullrich. The lean 4 theorem prover and programming language. In *Automated Deduction (CADE-28)*, 2021.